

课程笔记

S294-277 主书理论补充章 C: Underactuated 动力学、控制与规划基础

Codex

2026-04-16

Appendix C: Underactuated 课程的动力学、控制与规划基础

MIT Underactuated Spring 2024 适合作为这本书的理论骨架，尤其能补强 dynamics、optimal control、trajectory optimization 与 planning 的数学基础。

Ch. 1 - Fully-actuated vs Underactuated Systems

Underactuated Robotics

© Russ Tedrake, 2024 Last modified . How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT . They will be updated throughout the Spring 2024 semester. Lecture videos are available on YouTube .

Fully-actuated vs Underactuated Systems

Many robots today move far too conservatively, and accomplish only a fraction of the tasks and achieve a fraction of the performance that they are mechanically capable of. In some cases, we are still fundamentally limited by control technology which matured on rigid robotic arms in structured factory environments, where large actuators could be used to "shape" the dynamics of the machine to achieve precision and repeatability. The study of underactuated robotics focuses on building control systems which instead exploit the natural dynamics of the machines in an attempt to achieve extraordinary performance in terms of speed, efficiency, or robustness.

In the last few years, controllers designed using machine learning have showcased the potential power of optimization-based control, but these methods do not yet enjoy the sample-efficiency nor algorithmic reliability that we've come to expect from more mature control

视频作者/频道: MIT Underactuated / Russ Tedrake

发布日期: Spring 2024

视频时长: 课程级补充章

视频链接: <https://underactuated.csail.mit.edu/Spring2024/schedule.html>

目录

1 本章定位与主书关系	2
1.1 为什么 underactuated 特别重要	2
1.2 本章小结	2
2 机械系统、欠驱动与简单摆	3
2.1 一个统一的机械系统表示	3
2.2 简单摆为什么是第一课	3
2.3 能量视角	4
2.4 本章小结	4
3 动态规划、最优控制与 LQR	4
3.1 动态规划与 Bellman 递推	4
3.2 为什么 RL 会落到这里	4
3.3 LQR: 局部最优控制的基石	5
3.4 LQR 与时间变系统	5
3.5 本章小结	5
4 典型欠驱动系统: Acrobot、Cart-Pole 与 Quadrotor	5
4.1 为什么要研究模型系统	5
4.2 Acrobot	6
4.3 Cart-Pole 与 Quadrotor	6
4.4 本章小结	6
5 Lyapunov、轨迹优化与运动规划	6
5.1 Lyapunov 分析与保证	6
5.2 区域吸引域与 SOS	6
5.3 轨迹优化: 从全局最优到单条轨迹	7
5.4 轨迹优化与 LQR 的关系	7
5.5 Sampling-based motion planning	7
5.6 本章小结	7
6 本章小结	7
7 总结与延伸	8

Appendix C: Underactuated 课程的动力学、控制与规划基础

MIT Underactuated Spring 2024 适合作为这本书的理论骨架，尤其能补强 dynamics、optimal control、trajectory optimization 与 planning 的数学基础。

Ch. 1 - Fully-actuated vs Underactuated Systems

Underactuated Robotics

© Russ Tedrake, 2024 Last modified . How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT . They will be updated throughout the Spring 2024 semester. Lecture videos are available on YouTube .

Fully-actuated vs Underactuated Systems

Many robots today move far too conservatively, and accomplish only a fraction of the tasks and achieve a fraction of the performance that they are mechanically capable of. In some cases, we are still fundamentally limited by control technology which matured on rigid robotic arms in structured factory environments, where large actuators could be used to "shape" the dynamics of the machine to achieve precision and repeatability. The study of underactuated robotics focuses on building control systems which instead exploit the natural dynamics of the machines in an attempt to achieve extraordinary performance in terms of speed, efficiency, or robustness.

In the last few years, controllers designed using machine learning have showcased the potential power of optimization-based control, but these methods do not yet enjoy the sample-efficiency nor algorithmic reliability that we've come to expect from more mature control

图 1: 课程官方材料中的代表性页面，用于帮助读者在进入本讲正文前建立整体上下文。

1 本章定位与主书关系

本章不是 S294-277 主线讲义中的某一节视频转写，而是主书的理论补充章。它的源头来自 MIT Underactuated Spring 2024 的官方 notes、schedule 页面，以及该课程的官方 YouTube playlist。官方 notes 本身就明确写到 “Lecture videos are available on YouTube”，因此这里的内容可以同时作为可读的课程附录和可追溯的官方补充材料。

对于主书《Robots That Learn》来说，这一章承担的是“理论骨架”的角色：主书前半部分更强调学习、示范、局部技能、视觉模仿和长程规划，而 underactuated 课程提供的是把这些内容写成动力学系统、优化问题、稳定性条件和规划问题的数学底座。换句话说，这一章回答的是“为什么学习型机器人方法会变成这些形式”，而不是只告诉读者“有什么方法”。

这一章的核心链条

动力学建模 → 动态规划与最优控制 → LQR 与局部稳定性 → Lyapunov 分析 → 轨迹优化与运动规划

这条链条基本就是主书中 control、planning、RL、policy search 和 imitation learning 之间的共同语言。

1.1 为什么 underactuated 特别重要

欠驱动系统（underactuated system）是指输入维度少于自由度维度的系统。对于机器人而言，这并不是边缘情形，而是常态：腿足机器人、柔性系统、悬挂系统、飞行器、带接触的操作系统，几乎都不可避免地出现欠驱动。Underactuated 课程的价值在于，它不是只把欠驱动当作一个困难，而是把它当作一种可以利用的结构。

1.2 本章小结

本章的目标不是替代主书内容，而是把主书中学习、控制和规划的经验性方法，放回到机械系统与最优控制的严格语境中。读者如果把这一章吃透，就会更容易理解为什么后续的行为克隆、视

觉模仿、长程规划和语言条件策略会自然地落在这些数学形式上。

2 机械系统、欠驱动与简单摆

2.1 一个统一的机械系统表示

最常见的机械系统可以写成

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{B}\mathbf{u}.$$

这里：

- $\mathbf{q}$ 是广义坐标 (generalized coordinates)；
- $\mathbf{M}(\mathbf{q})$ 是质量矩阵 (mass matrix)；
- $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ 是科氏/离心项；
- $\mathbf{g}(\mathbf{q})$ 是重力项；
- $\mathbf{u}$ 是控制输入；
- $\mathbf{B}$ 是输入分配矩阵。

当输入数目少于自由度数目时， $\mathbf{B}$ 不可能在所有方向上都独立施力，这就是欠驱动的本质。Underactuated notes 反复强调的一点是：很多机器人系统并不是“控制不够强”，而是“可用控制方向本来就有限”，所以真正的任务是利用自然动力学，而不是试图完全压制它。

2.2 简单摆为什么是第一课

简单摆 (simple pendulum) 之所以是经典例子，不只是因为它最简单，而是因为它把机器人控制里最常见的几个概念一次性摆到桌面上：非线性、能量、相图、平衡点、稳定性、摆起 (swing-up) 和局部线性化。

若将摆角记为 θ ，则无控制时可以写成

$$\ddot{\theta} = \frac{g}{\ell} \sin \theta,$$

如果加上关节力矩 u ，则典型形式可写成

$$\ddot{\theta} = \frac{g}{\ell} \sin \theta + \frac{1}{m\ell^2} u.$$

这里：

- g 是重力加速度；
- ℓ 是连杆长度；
- m 是摆球质量；
- u 是控制力矩。

这个系统的相图 (phase portrait) 清楚地告诉我们：非线性机器人系统的关键，不是某个孤立轨迹，而是整个状态空间里的流形结构。主书中的 Acrobot、Cart-Pole、视觉控制中的姿态调整，本质上都能在这个层面理解。

2.3 能量视角

对摆而言，机械能可以写成

$$E(\theta, \dot{\theta}) = \frac{1}{2}m\ell^2\dot{\theta}^2 + mg\ell(1 - \cos\theta).$$

如果目标是把摆从下垂位置摆到竖直平衡点附近，那么一个自然的思路不是“直接消灭所有动力学”，而是先把系统的能量调到正确的水平，再用局部反馈把误差压住。这个思想后来会以 energy shaping、swing-up、轨迹优化和 policy search 等不同名字反复出现。

从简单摆到机器人

简单摆不是玩具，而是一个压缩版的机器人动力学问题。它足够简单，能让我们看清非线性；它又足够丰富，能把能量、稳定性和控制设计联系起来。

2.4 本章小结

欠驱动不是控制理论的边角料，而是机器人控制的核心结构之一。简单摆把“机械能如何被控制”“局部线性化为什么有用”“全局非线性为什么难”这三件事都讲得非常清楚。

3 动态规划、最优控制与 LQR

3.1 动态规划与 Bellman 递推

动态规划（dynamic programming）把控制设计写成一个递推优化问题。对于离散时间系统

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k),$$

有限时域目标可写为

$$J(\pi) = \phi(\mathbf{x}_T) + \sum_{k=0}^{T-1} \ell(\mathbf{x}_k, \mathbf{u}_k),$$

其中 ℓ 是阶段代价， ϕ 是终端代价， π 表示控制策略。

对应的价值函数（value function）满足 Bellman 方程

$$V_k(\mathbf{x}) = \min_{\mathbf{u}} \{ \ell(\mathbf{x}, \mathbf{u}) + V_{k+1}(f(\mathbf{x}, \mathbf{u})) \}.$$

这里的意思非常简单：最优控制问题可以拆成“当前一步的代价”加上“未来最优剩余代价”。

连续时间极限下，会得到 Hamilton–Jacobi–Bellman (HJB) 方程：

$$\partial_t V(t, \mathbf{x}) + \min_{\mathbf{u}} \{ \ell(\mathbf{x}, \mathbf{u}) + \nabla_{\mathbf{x}} V(t, \mathbf{x})^\top f(\mathbf{x}, \mathbf{u}) \} = 0.$$

这就是 underactuated notes 里把 optimal control、reinforcement learning 和 computational optimization 串在一起的数学核心。

3.2 为什么 RL 会落到这里

主书里讲到 reinforcement learning（强化学习）时，如果只把它看成神经网络训练，很容易失去结构感。Underactuated 课程把它直接放回 optimal control：很多 RL 方法，本质上是在近似求解价值函数、策略函数或者 HJB/Bellman 递推。这样一来，探索、价值传播、策略优化和约束处理就都有了统一解释。

3.3 LQR: 局部最优控制的基石

线性二次调节器 (linear quadratic regulator, LQR) 是最经典的局部控制器之一。在线性化系统

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$$

上, 若目标是最小化二次型代价

$$J = \int_0^\infty (\mathbf{x}^\top Q \mathbf{x} + \mathbf{u}^\top R \mathbf{u}) dt,$$

则最优控制律为

$$\mathbf{u} = -K\mathbf{x}, \quad K = R^{-1}B^\top S.$$

这里 S 满足代数 Riccati 方程

$$A^\top S + SA - SBR^{-1}B^\top S + Q = 0.$$

这组公式的重要性在于, 它给出了机器人控制里最常见的一条工程路径: 先在目标附近线性化, 再用 LQR 做局部稳定化。它不是全局答案, 但它是可解释、可实现、最容易和 trajectory optimization 结合的局部答案。

3.4 LQR 与时间变系统

当控制目标不是一个固定平衡点, 而是一条轨迹时, LQR 会变成 time-varying LQR (TVLQR)。此时反馈律通常写成

$$\mathbf{u}(t) = \mathbf{u}^*(t) - K(t)(\mathbf{x}(t) - \mathbf{x}^*(t)),$$

其中 $(\mathbf{x}^*(t), \mathbf{u}^*(t))$ 来自轨迹优化, $K(t)$ 则来自沿轨迹线性化后的 Riccati 方程。这个结构在机器人系统里极其常见: 轨迹优化负责找“中间那条路”, LQR 负责把这条路稳住。

3.5 本章小结

动态规划提供了最优性原则, HJB 把它变成连续时间控制的方程形式, 而 LQR 则给出了最重要的可计算特殊情形。对于主书读者来说, 最值得记住的是: 很多看似“学习算法”的东西, 理论上都可以理解为对最优控制问题的近似求解。

4 典型欠驱动系统: Acrobot、Cart-Pole 与 Quadrotor

4.1 为什么要研究模型系统

Underactuated notes 反复使用 Acrobot、Cart-Pole 和 Quadrotor 这样的 canonical systems, 不是因为它们足够“真实”, 而是因为它们足够“代表性强”。模型系统的价值在于: 它们把某一类困难提纯出来, 让控制方法的结构一目了然。

系统	主要困难	典型任务	课程中的作用
Acrobot	关节欠驱动、摆起难	swing-up + balance	非线性控制与能量法
Cart-Pole	直线平移耦合姿态	平衡与摆起	DP / LQR / swing-up
Quadrotor	姿态与平移强耦合	稳定、轨迹跟踪	三维控制与规划

表 1: 三类典型欠驱动模型的角色。

4.2 Acrobot

Acrobot 是一个竖直平面内的两连杆机械臂，只有肘部有驱动，肩部没有驱动。它的任务通常是 swing-up：先把机械臂摆到竖直附近，再用局部反馈稳定在倒立附近。这个问题在 underactuated 语境里非常经典，因为它逼着我们处理“如何利用重力和惯性，而不是对抗它们”。

4.3 Cart-Pole 与 Quadrotor

Cart-Pole 是另一类教科书式例子：小车可以平移，杆子可以摆动，目标通常是把杆子立起来并保持平衡。它最重要的意义在于，系统既有可直接操控的平移自由度，又有必须借助耦合才能控制的姿态自由度。

Quadrotor 则把欠驱动思想推广到三维飞行。飞行器的推力方向受到机体姿态约束，因此平移控制和姿态控制并不是完全独立的。它是很好的例子，说明 underactuated 并不意味着“少几个电机”这么简单，而是意味着控制方向和任务方向存在结构性不匹配。

4.4 本章小结

模型系统的目的不是简化成玩具，而是把复杂机器人问题里最关键的 control structure 单独拿出来。Acrobot、Cart-Pole 和 Quadrotor 共同构成了欠驱动机器人控制最常用的入门字典。

5 Lyapunov、轨迹优化与运动规划

5.1 Lyapunov 分析与保证

Lyapunov 分析的关键问题不是“最优不最优”，而是“能不能保证”。对于系统

$$\dot{\mathbf{x}} = f(\mathbf{x}),$$

如果我们能找到一个函数 $V(\mathbf{x})$ 使得

$$V(\mathbf{x}) > 0, \quad \dot{V}(\mathbf{x}) = \nabla V(\mathbf{x})^\top f(\mathbf{x}) < 0$$

(在平衡点之外)，那么系统就在该区域内渐近稳定。

这里的符号含义是：

- V 是 Lyapunov 函数；
- 正定性表示它像“能量”一样非负；
- $\dot{V} < 0$ 表示系统沿着轨迹持续消耗该“能量”。

对机器人而言，Lyapunov 方法的重要性在于它能把“控制是否有效”转化成可验证的数学陈述。主书里很多 local controller、stability certificate 和 region-of-attraction 的想法，都可以从这里理解。

5.2 区域吸引域与 SOS

实际系统往往不会只关心一个孤立平衡点，而是关心它的吸引域 (region of attraction, ROA) 有多大。若把 V 选成多项式函数，并把系统动力学写成多项式形式，就可以借助 sums-of-squares (SOS) 优化去搜索一个候选 Lyapunov 函数，从而估计 ROA。

这类方法在 underactuated notes 中的定位很明确：它们不是为了替代工程控制，而是为了给工程控制提供“可证明的安全边界”。

5.3 轨迹优化：从全局最优到单条轨迹

轨迹优化 (trajectory optimization) 把最优控制问题从“全状态空间的反馈律”收缩为“一条可行轨迹”。如果目标是寻找一条从初始状态到终端状态的轨迹，那么可以写成

$$\min_{\mathbf{x}(\cdot), \mathbf{u}(\cdot)} \int_0^T \ell(\mathbf{x}(t), \mathbf{u}(t)) dt + \phi(\mathbf{x}(T))$$

满足

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{x}(0) = \mathbf{x}_0.$$

这一步的意义非常大：它把难以处理的高维反馈设计，转成有限维非线性规划 (nonlinear programming, NLP)。常见的数值做法包括 direct shooting、multiple shooting 和 collocation。

5.4 轨迹优化与 LQR 的关系

轨迹优化给出名义解 $(\mathbf{x}^*(t), \mathbf{u}^*(t))$ ，然后可以沿着该轨迹线性化并设计 TVLQR 稳定器。于是，一个很实用的机器人控制栈就出现了：

$$\text{trajectory optimization} \Rightarrow \text{nominal plan} \Rightarrow \text{TVLQR stabilization}.$$

这正是 underactuated 思路 and 主书后半部分的长程规划、模仿学习、局部策略之间最重要的桥梁之一。

5.5 Sampling-based motion planning

运动规划 (motion planning) 在 underactuated notes 中并不只指轨迹优化，也包括 sampling-based motion planning。这里的直觉是：如果状态空间太大、障碍太复杂、局部最优太多，那就不要试图用连续优化一次性打穿问题，而是通过随机采样逐步构建可行路径。

最典型的算法是 PRM (probabilistic roadmap) 和 RRT (rapidly-exploring random tree)。它们特别适合几何复杂、约束复杂、局部搜索容易卡住的问题。对于有动力学约束的机器人，还要进一步进入 kinodynamic planning，即在规划时显式考虑动力学可达性。

轨迹优化与采样规划的分工

轨迹优化擅长局部连续改进，采样规划擅长全局可达性搜索。欠驱动系统的规划通常需要两者结合：先找得到，再走得稳。

5.6 本章小结

Lyapunov、轨迹优化和采样式规划并不是三套互不相关的工具，而是同一条控制链上的不同环节：Lyapunov 提供保证，轨迹优化提供可计算的名义方案，采样式规划提供全局搜索能力。

6 本章小结

这章可以用一句话概括：欠驱动机器人之所以难，不是因为它们“太复杂”，而是因为它们必须同时尊重动力学、控制约束和优化结构。

从学习主书的角度看，这一章最重要的贡献有三点：

- 它把 RL 放回 optimal control 的统一框架里；
- 它把局部稳定化和全局规划连接起来；
- 它说明了为什么简单模型、Lyapunov 证书和轨迹优化会反复出现在机器人文献中。

7 总结与延伸

如果把主书《Robots That Learn》与 Underactuated Spring 2024 并读，读者会得到一张非常完整的地图：

- 主书回答“机器人如何从数据和示范中学习行为”；
- 本附录回答“这些行为为什么必须服从动力学与稳定性结构”；
- 两者合起来，才能真正理解学习、控制和规划之间的边界与互补。

后续最值得继续阅读的顺序是：

- 先回看官方 schedule 中的 Ch. 1, 2, 7, 8, 9, 10, 12；
- 再回看主书中对应的 RL、behavior cloning、visual imitation、locomotion 和 long-horizon planning 章节；
- 最后把 trajectory optimization、TVLQR、Poincaré map 和 policy search 这些概念放到同一张图里理解。

这也是本章真正想传达的底层观点：学习型机器人方法并不是脱离控制理论的替代品，而是控制理论在更复杂、更多样、更多模态任务上的延伸。